CSA0604 – DESIGN AND ANALYSIS OF ALGORITHMS

LAB EXPERIMENTS -SESSION 4

NAME : Shree Ranjane K

REG NO: 192471052

1. **Find All Large Groups in a String**

```
#include <stdio.h>
#include <string.h>

int main()
{
   char s[1000];
   int n, i, start, end, length;
   int found = 0;

   printf("Enter a string: ");
   scanf("%999s", s);

   n = strlen(s);

   printf("Large groups are:\n");

   i = 0;

   while (i < n)
   {
      start = i;

      // Find consecutive characters that are the same
      while (i < n && s[i] == s[start])
      {
         i++;
      }

      end = i - 1;
      length = end - start + 1;

      // Check whether the group has at least 3 characters
      if (length >= 3)
      {
```

```
            printf("[%d,%d]\n", start, end);
            found = 1;
        }
    }

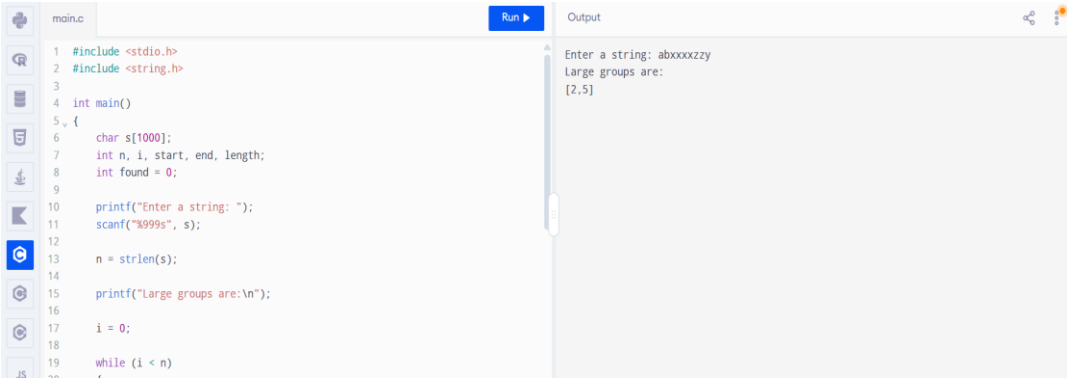
    if (found == 0)
    {
        printf("No large groups found.\n");
    }

    return 0;
}
```

Output 1
Enter a string: abbxxxxzzy
Large groups are:
[3,6]

Output 2
Enter a string: abc
Large groups are:
No large groups found.

Output 3
Enter a string: aaabbbcccc
Large groups are:
[0,2]
[3,5]
[6,9]



Time Complexity: \(O(n)\), where \(n\) is the length of the string.
Space Complexity: \(O(n)\) for storing the input string.

2. Heading: Implementation of Conway's Game of Life Using C

```
#include <stdio.h>

void gameOfLife(int m, int n, int board[m][n])
{
   int i, j, x, y, live;

   int dir[8][2] = {
      {-1, -1}, {-1, 0}, {-1, 1},
      { 0, -1},          { 0, 1},
      { 1, -1}, { 1, 0}, { 1, 1}
   };

   // First pass: store the next state temporarily
   for (i = 0; i < m; i++)
   {
      for (j = 0; j < n; j++)
      {
         live = 0;

         // Count live neighbors
         for (int k = 0; k < 8; k++)
         {
            x = i + dir[k][0];
            y = j + dir[k][1];

            if (x >= 0 && x < m &&
               y >= 0 && y < n)
            {
               if (board[x][y] == 1 ||
```

```
                board[x][y] == 2)
            {
                live++;
            }
        }
    }

    // 2 represents live -> dead
    if (board[i][j] == 1)
    {
        if (live < 2 || live > 3)
            board[i][j] = 2;
    }
    // 3 represents dead -> live
    else
    {
        if (live == 3)
            board[i][j] = 3;
    }
  }
}

// Second pass: update all cells
for (i = 0; i < m; i++)
{
  for (j = 0; j < n; j++)
  {
    if (board[i][j] == 2)
        board[i][j] = 0;
    else if (board[i][j] == 3)
```

```
                board[i][j] = 1;
            }
        }
    }

int main()
{
    int m, n, i, j;

    printf("Enter rows and columns: ");
    scanf("%d %d", &m, &n);

    int board[m][n];

    printf("Enter the board elements (0 or 1):\n");

    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &board[i][j]);
        }
    }

    gameOfLife(m, n, board);

    printf("Next generation:\n");

    for (i = 0; i < m; i++)
    {
```

```
        for (j = 0; j < n; j++)
        {
            printf("%d ", board[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Sample Input

4 3

0 1 0

0 0 1

1 1 1

0 0 0

Sample Output

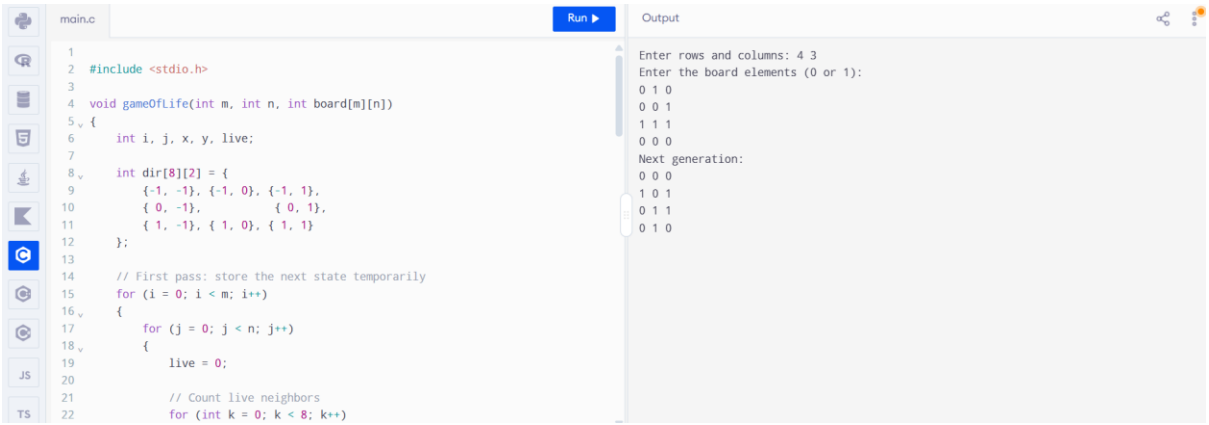
Next generation:

0 0 0

1 0 1

0 1 1

0 1 0



Time Complexity: \(O(m \times n)\), because each cell checks eight neighbors.

Auxiliary Space Complexity: \(O(1)\), because the board is updated in place using temporary state values.

3.Champagne Tower Using Dynamic Programming in C

```
#include <stdio.h>


double champagneTower(int poured, int query_row, int query_glass)
{
   double dp[101][101] = {0};


   dp[0][0] = poured;


   for (int i = 0; i < query_row; i++)
   {
      for (int j = 0; j <= i; j++)
      {
         if (dp[i][j] > 1.0)
         {
            double excess = (dp[i][j] - 1.0) / 2.0;


            dp[i + 1][j] += excess;
```

```
                dp[i + 1][j + 1] += excess;
            }
        }
    }

    if (dp[query_row][query_glass] > 1.0)
        return 1.0;

    return dp[query_row][query_glass];
}

int main()
{
    int poured, query_row, query_glass;

    printf("Enter poured, query_row and query_glass: ");
    scanf("%d %d %d", &poured, &query_row, &query_glass);

    double result = champagneTower(poured, query_row, query_glass);

    printf("Output: %.5f\n", result);

    return 0;
}
```

Sample Output 1

Input:

1 1 1

Output:

Output: 0.00000

Sample Output 2

Input:

2 1 1

Output:

Output: 0.50000



Time Complexity: \(O(r^2)\), where \(r\) is query_row.

Space Complexity: \(O(r^2)\), where \(r\) is query_row.